

BACCALAURÉAT GÉNÉRAL

ÉPREUVE D'ENSEIGNEMENT DE SPÉCIALITÉ

Session 2025

NUMÉRIQUE ET SCIENCES INFORMATIQUES

Amérique du Nord 2025 – Jour 2

DURÉE DE L'ÉPREUVE: 3 heures 30

*Ce sujet comporte 7 pages numérotées de 1/7 à 7/7
La dernière page comporte une annexe à remettre complétée de votre nom et classe.
Le candidat doit traiter les trois exercices proposés.*

L'utilisation d'une calculatrice n'est pas autorisée

Exercice 1 ▶

/6 points

1. a a pour code ASCII 97, soit 01100001 en binaire
2. `replique([0, 0, 1, 0, 1])` renvoie `[0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1]`
3. Fonction `nb_occurences`

</> Code Python

```
1 def nb_occurences(tab, i):
2     nb_occ = {}
3     for j in range(3 * i, 3 * (i + 1)):
4         x = tab[j]
5         if x in nb_occ:
6             nb_occ[x] += 1           # Ligne 14
7         else:
8             nb_occ[x] = 1           # Ligne 16
9     return nb_occ
```

4. Fonction `majorite`

</> Code Python

```
1 def majorite(dict):
2     cle_max = None
3     valeur_max = -1
4     for cle in dict.keys():
5         if dict[cle] > valeur_max:   # Ligne 11
6             valeur_max = dict[cle]   # Ligne 12
7             cle_max = cle            # Ligne 13
8     return cle_max
```

5. Détection d'erreur dans un tableau. Le bit erroné est le bit

- première Ligne
- deuxième colonne

6. Fonction `erreur_colonne`

</> Code Python

```
1 def erreur_colonne(mat):
2     for col in range(len(mat[0])):
3         somme = 0
4         for ligne in range(len(mat)):
5             somme += mat[ligne][col]
6         if somme % 2 == 1: # Parité impaire = erreur
7             return col
```

8. Code reçu: 1010000. Le code le plus proche qui ne diffère que par un bit dans le tableau de Hamming est 1110000 soit le mot 1000
9. Fonction `corriger_erreur`

</> Code Python

```
1 def corriger_erreur(code_recu):
2     # si le code est correct, on le retourne
3     if code_recu in hamming_4_7:
4         return code_recu
5     else:
6         # copie du code reçu
7         code = [bit for bit in code_recu]      # Ligne 17
8         # on teste chaque bit
9         for indice in range(7):
10            # on inverse le bit
11            code[indice] = (code[indice] + 1) % 2    # Ligne 20
12            # on teste si le code est correct
13            if code in hamming_4_7:
14                return code
15            # sinon on reinitialise le bit à sa valeur d'origine
16            else:
17                code[indice] = code_recu[indice]      # Ligne 25
```

10. Il comporte 2^7 feuilles, soit 128 feuilles.

11. Fonction decode

</> Code Python

```
1 def decode(arbre, code, i):
2     # si on est arrivé à une feuille, retourner l'étiquette
3     if i == len(code):
4         return arbre.etiquette
5     # on est sur un noeud
6     # on suit la branche correspondant à la valeur du bit
7     if code[i] == 0:
8         # on suit la branche gauche
9         # on incrémente le bit
10        return decode(arbre.gauche, code, i+1)    # Ligne 12
11    if code[i] == 1:
12        # on suit la branche droite
13        # on incrémente le bit
14        return decode(arbre.droit, code, i+1)      # Ligne 14
```

Exercice 2 ▶

/6 points

1. Pour 8 bonbons:
 - On mange le bonbon 0. Il reste [1, 2, 3, 4, 5, 6, 7]
 - On mange le bonbon 3. Il reste [1, 2, 4, 5, 6, 7]
 - On mange le bonbon 6. Il reste [1, 2, 4, 5, 7]
 - On mange le bonbon 2. Il reste [1, 4, 5, 7]
 - On mange le bonbon 7. Il reste [1, 4, 5]
 - On mange le bonbon 5. Il reste [1, 4]
 - On mange le bonbon 1. Il reste [4]. Il restera donc le bonbon 4
2. Une erreur `NameError` signifie qu'un nom de variable n'est pas défini. Ici, `true` n'existe pas en Python.
Correction: `collier = [True for i in range(8)]` (avec T majuscule)
3. Fonction `dernier`

```
</> Code Python
1 def dernier(n):
2     collier = [True for i in range(n)]           # Ligne 2
3     indice = 0
4     collier[indice] = False
5     for etape in range(n-1):
6         nb_bonbons_vus = 0
7         while nb_bonbons_vus < 3:                 # Ligne 7
8             indice += 1
9             if indice >= n:                         # Ligne 9
10                 indice = 0
11                 if collier[indice]:                # Ligne 11
12                     nb_bonbons_vus += 1
13                 collier[indice] = False            # Ligne 13
14     return indice
```

4. Acronyme adapté à File: FIFO (First In First Out)
5. Résultat: (Tête) 3 4 1 2 (Queue)
6. Fonction `dernier_file`

```
</> Code Python
1 def dernier_file(n):
2     f = File()
3     for i in range(n):
4         f.enqueue(i)
5     f.dequeue()
6     while not f.est_vide():
7         for _ in range(2):                         # Défiler 2 bonbons et les remettre
8             if not f.est_vide():
9                 bonbon_defile = f.dequeue()
10                f.enqueue(bonbon_defile)
11            if not f.est_vide():                     # Manger le 3ème bonbon
12                dernier = f.dequeue()
13                if f.est_vide():
14                    return dernier
15     return dernier
```

7. Terminologie POO: Les variables `pred`, `valeur` et `succ` sont des attributs (ou propriétés) de la classe.

8. A la fin de la séquence: (`self.pred.valeur`, `self.valeur`, `self.succ.valeur`)

- 2 0 1
- 0 1 0
- 1 2 0

On a donc:

- `zero.succ.valeur`: la valeur du successeur de 0, soit 1
- `un.succ.succ.pred.valeur`: la valeur du successeur de 1, soit 0

9. Fonction `creer_collier`

```
</> Code Python
1 def creer_collier(n):
2     # Création du premier bonbon
3     premier = Bonbon(0)
4     actuel = premier
5     for i in range(1, n):
6         # création du nouveau bobon
7         nouveau = Bonbon(i)           # Ligne 5
8         # on accroche les successeurs et predecesseurs
9         actuel.succ = nouveau         # Ligne 6
10        nouveau.pred = actuel         # Ligne 7
11        # on se déplace dans le collier
12        actuel = nouveau
13    # on referme le collier
14    actuel.succ = premier              # Ligne 9
15    premier.pred = actuel             # Ligne 10
16    return premier
```

10. 1 - 2 - 3 - Retour vers 1

11. Condition pour un seul bonbon

Proposition C: `bonbon.valeur == bonbon.succ.valeur` est vraie quand il ne reste qu'un bonbon.

12. Fonction `dernier_chaine`

```
</> Code Python
1 def dernier_chaine(n):
2     bonbon = creer_collier(n)
3     while bonbon.succ != bonbon:      # Ligne 3
4         bonbon.pred.succ = bonbon.succ # Ligne 4
5         bonbon.succ.pred = bonbon.pred # Ligne 5
6         bonbon = bonbon.succ.succ.succ # Ligne 6
7     return bonbon.valeur
```

Exercice 3 ▶

/8 points

1. Plusieurs passagers peuvent réserver le même vol, donc `id_vol` peut apparaître plusieurs fois.
2. Clé primaire: (`id_vol`, `id_passager`) - couple unique identifiant chaque réservation.
3. Une clé étrangère assure l'intégrité référentielle en garantissant qu'une valeur référence une clé primaire existante dans une autre table.
4. Affiche les `id_vol` de la table `vol` qui ont un aéroport d'arrivée 'CDG' (Charles de Gaulle).

```
AI0015
AI0258
AI0292
```

5. Villes destinations depuis CDG:

```
SELECT DISTINCT ville
FROM aeroport
JOIN vol ON aeroport.id_aeroport = vol.aeroport_arr
WHERE vol.aeroport_dep = 'CDG';
```

6. Mise à jour de la distance totale:

```
UPDATE passager
SET d_totale = d_totale + 6
WHERE id_passager = 5;
```

7. Correction de l'insertion. AI0256 existe déjà comme clé primaire.
Correction: Utiliser un nouvel identifiant:

```
INSERT INTO vol VALUES ('AI0300', 'CDG', 'YUL', 6);
```

8. La valeur de cette expression est 10
9. Fonction `vol_direct`

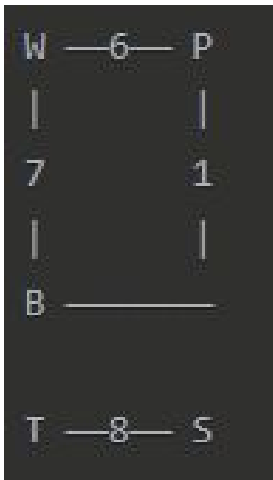
</> Code Python

```
1 def vol_direct(graphe, ville1, ville2):
2     if ville1 not in graphe.keys() or ville2 not in graphe.keys():
3         return False
4     return ville2 in graphe[ville1].keys()
```

10. Fonction `liste_villes_proches`

</> Code Python

```
1 def liste_villes_proches(graphe, ville, d_max):
2     villes_proches = []
3     for voisine, distance in graphe[ville].items():
4         if distance <= d_max:
5             villes_proches.append(voisine)
6     return villes_proches
```

11. Graphe `droit_devant`

12. Proposition A est correcte : le graphe `AirInfo` est connexe. Le graphe `Droidevant` n'est pas connexe (T-S isolés de W-P-B).

13. La fonction `parcours` est récursive car elle s'appelle elle-même à la ligne 12.

14. Contenu des variables:

- `visitees1` contient ['W', 'P', 'T', 'B', 'S']
- `visitees2` contient ['W', 'P', 'B']

15. Type de parcours. Proposition C: parcours en profondeur (DFS).

16. Fonction `est_connexe`

Code Python

```

1 def est_connexe(graphe):
2     depart = ville_arbitraire(graphe)
3     visitees = []                                     # Ligne 4
4     parcours(graphe, visitees, depart)
5     return len(visitees) == len(graphe)             # Ligne 6
  
```

17. Affichage de la fonction `mystere`. Les différents chemins allant de W à B sont:

- ['W', 'P', 'T', 'B'] 25
- ['W', 'P', 'S', 'T', 'B'] 40

18. La fonction `mystere` trouve et affiche tous les chemins possibles entre deux villes avec leur coût total, en évitant les cycles.